

30-Day Machine Learning Internship Workplan

Welcome! This is your roadmap for the next 30 days. It's built around one idea: **every week, you ship something real**. By the end of Week 4, you'll have a live, working ML demo that anyone can open and use — not just a notebook full of code.

No prior ML experience is assumed. Each task in Weeks 1–2 comes with a short concept explainer, so you'll learn things right before you use them, not weeks in advance.

What you'll build: you'll pick a project on Day 1, work through it solo, and end with a deployed model you can demo live. Model accuracy matters less than having something that actually works and that you fully understand.

STAGE 01
EXPLORE

STAGE 02
BASELINE

STAGE 03
ITERATE

STAGE 04
SHIP

WK. 01 Get Comfortable + Understand Your Data

PIPELINE STAGE
01 · EXPLORE

BY FRIDAY, YOU'LL HAVE

A cleaned, documented dataset and an exploratory data analysis (EDA) notebook, which you'll walk someone through in a 10-minute demo.

Each day pairs a short learning block (1–2 hrs) with hands-on practice the same day — you'll use what you just learned immediately, not weeks later.

DAY	FOCUS	TASK	WHAT YOU'LL HAVE
01	Setup + orientation	Set up your environment (Python, Jupyter/Colab, Git), get repo access. Learn: what ML is, supervised vs. unsupervised, the standard workflow (data → model → evaluate → deploy). Pick your project.	Working dev environment, first commit, and a 1-paragraph pitch of what you're predicting and why it's useful
02	Python for data + getting your dataset	Learn: pandas basics (load, filter, groupby). Download and load your chosen dataset.	Raw dataset loaded in a notebook, plus a short doc on where it's from, its license, and what's in it
03	Data cleaning	Learn: handling missing values, duplicates, and mismatched types. Apply it to your dataset.	<code>clean_data.py</code> script
04	Exploring your data	Learn: what makes a chart or insight actually useful. Build summary stats, distributions, correlations, visualizations.	EDA notebook with at least 5 charts and written observations
05	Present your findings	Package everything up and present it.	10-min demo: dataset overview + 3 key insights + open questions

BEFORE YOU MOVE ON, MAKE SURE

Someone unfamiliar with your dataset could understand it just from your notebook. And you can explain, in plain language, what you're trying to predict and why.

BY FRIDAY, YOU'LL HAVE

A working baseline model and a clear way to measure how good it is.

DAY	FOCUS	TASK	WHAT YOU'LL HAVE
06	Framing the problem	Learn: classification vs. regression, what a "target variable" is. Define your task precisely, pick your target variable, define one success metric.	1-page problem statement
07	Preparing features	Learn: why models need numbers, not text, and what scaling/encoding do. Encode categoricals, scale/normalize, split into train/test sets.	<code>feature_pipeline.py</code>
08	Your first model	Learn: what scikit-learn is, <code>.fit()</code> / <code>.predict()</code> . Train the simplest reasonable model (logistic/linear regression or decision tree).	Baseline model script + saved model file
09	Measuring performance	Learn: what accuracy, F1, and RMSE actually mean and when each applies. Build an evaluation script, confusion matrix or residual plot.	<code>evaluate.py</code> + metrics report
10	Present your baseline	Present your results and explain your metric in plain language.	10-min demo: your baseline number + why you chose that metric

BEFORE YOU MOVE ON, MAKE SURE

You have one clear number that everything you build next needs to beat, and you can explain what that number actually means without jargon.

BY FRIDAY, YOU'LL HAVE

One improved model that beats your Week 2 baseline, locked in and ready to deploy.

Your end goal is a working demo, not the best possible model — so this week is intentionally light. One clean improvement beats three half-finished experiments.

DAY	FOCUS	TASK	WHAT YOU'LL HAVE
11	Pick your next model	Learn: random forest / gradient boosting basics — why they usually beat a plain baseline. Choose one candidate model type.	A short note on what you picked and why
12	Train it	Train your improved model and compare it to your baseline on the same metric.	Model file + logged metrics
13	Light feature tweaks	Try 1–2 feature improvements — only keep them if they clearly help. Don't chase small gains.	Updated features (if it helped)
14	Tune it + lock it in	Do some light hyperparameter tuning (a few manual tries, or a basic grid search — not exhaustive). Save your final model.	Final model file (<code>.pkl</code> or <code>.joblib</code>)
15	Present your progress	Compare your final model to your baseline.	Comparison table (baseline vs. final) + demo

BEFORE YOU MOVE ON, MAKE SURE

Your model is good enough to deploy. It doesn't need to be perfect — it needs to be done.

BY THE END OF THIS WEEK, YOU'LL HAVE

A live, working demo — something you can hand someone a link to and watch them use it, with no explanation needed.

This is the most important week, so it gets the most time. You'll use **Streamlit** (fastest way to a working UI) or a **Flask/FastAPI** endpoint if you want API experience.

DAY	FOCUS	TASK	WHAT YOU'LL HAVE
16	Package your model	Learn: what "inference" means and why it's kept separate from training. Wrap your final model in a clean, reusable predict function.	<code>predict.py</code> with a clean input/output interface
17	Build your interface (part 1)	Learn: what Streamlit/Flask does. Scaffold a minimal app that loads your model and takes user input.	Skeleton app running on your machine
18	Build your interface (part 2)	Wire up predictions, display results, add basic input validation.	Fully working demo, running locally
19	Deploy it live	Deploy to a free host (Streamlit Community Cloud, Render, Hugging Face Spaces, etc.).	A live public URL you can share with anyone
20	Stress-test it	Test with weird or bad inputs, fix crashes, handle errors gracefully so it doesn't break mid-demo.	Confirmed stable, no crashes on bad input
21	Write it up	Write a README: setup, usage, what your model can and can't handle.	Final <code>README.md</code>
22	Prep your final presentation	Build a short slide deck: the problem, your approach, your results, your demo link, limitations, what you'd do next.	Slide deck (max 8 slides)
FIN	Present live	Present your project end-to-end, demo the live app, take questions.	Live demo + Q&A

BEFORE YOU'RE DONE, MAKE SURE

Someone outside the project can open your link and get a real result — without you standing there explaining it.

Weekly Rhythm

DAILY · 15 MIN Standup: what you did yesterday, what's planned today, anything blocking you.	FRIDAY · 30 MIN Demo: show your week's actual output live — not a status update, a real working thing.	2x / WEEK · 30 MIN Office hours for getting unstuck on anything technical.
--	--	--

Pick Your Project (Day 1)

- Choose one clean, beginner-friendly public dataset — the goal is a smooth path to a deployed demo, not a research-grade dataset.
- Classification:** Titanic survival, customer churn, or heart disease prediction (Kaggle "Getting Started" datasets)
 - Regression:** House price prediction, used car price prediction (Kaggle beginner datasets)
 - Text (NLP-lite):** Spam/ham text classification, IMDB sentiment analysis (small, well-labeled datasets)
 - Images (optional, only if you're ahead of pace):** MNIST digit classification — this adds complexity that can eat into your deployment time, so only pick this if Weeks 1–3 felt easy

How You'll Be Evaluated

CRITERION	WEIGHT
Deployed demo works & is stable	30%
Code quality & reproducibility	20%
Data handling & EDA quality	15%
Model performance vs. baseline	15%
Documentation & clarity	10%
Final presentation & answering questions	10%

Tools You'll Use

- Python 3.x, Jupyter/Colab
- pandas, numpy, scikit-learn, matplotlib/seaborn
- Git/GitHub for version control
- **Streamlit** (recommended — fastest path to a working, deployable UI) or Flask/FastAPI if you want API experience
- Free deployment: Streamlit Community Cloud, Hugging Face Spaces, or Render